

Tool-Schema Compression Enables Agentic RAG Under Constrained Context Budgets

Furkan Sakizli

Independent Researcher

ORCID: 0009-0009-5975-5014

Abstract

Agentic RAG systems that equip language models with dozens to hundreds of tool definitions face a critical resource conflict: tool schemas consume the same context window needed for retrieval-augmented generation. We present the first systematic study of this *tool-context trade-off*, evaluating 14 models spanning 1.5B–32B local models plus one frontier API model across 6,566 controlled API calls at three context budgets (8K, 16K, 32K) with 28 tool definitions. Applying TSCG conservative-profile compression (44–50% schema token savings), we observe a **binary enablement effect**: at 8K tokens, JSON-schema tool definitions overflow the context window entirely, yielding near-zero EM (2.6% average), while compressed schemas restore RAG functionality with +20.5 pp average exact-match lift across all eight models (+24.7 pp among the six exhibiting full enablement). At 32K—where both formats fit—four of five tested models show $\Delta \leq 1$ pp, confirming the effect is purely budget-driven. External validation on HotpotQA (50 multi-hop questions) shows +48 pp EM under the same overflow scenario. Frontier scaling tests demonstrate that JSON schemas overflow at ~ 494 tools while compressed schemas remain operational beyond 800 tools. Our results establish tool-schema compression as a necessary infrastructure layer for agentic RAG in constrained-context deployments. All code, data, and checkpoints are publicly available.¹

1 Introduction

The convergence of tool-augmented language models (Schick et al., 2024; Patil et al., 2024) and retrieval-augmented generation (RAG; Lewis et al., 2020; Gao et al., 2024) has produced a new class of *agentic RAG* systems: agents that select tools, issue queries, and synthesize retrieved evidence

within a single context window. Production deployments routinely expose 20–100+ tools through protocols such as the Model Context Protocol (MCP; Anthropic, 2024), while simultaneously requiring space for system prompts, retrieval chunks, conversation history, and output generation.

This creates a fundamental **resource conflict**. Tool schemas—typically represented as verbose JSON Schema objects—can consume 300–500 tokens per tool. At 28 tools, the JSON-schema block alone requires $\sim 11,000$ tokens, which *exceeds* an 8K context window and substantially constrains retrieval at 16K (leaving room for only 6–11 chunks vs. 25–28 with compressed schemas). Yet the vast majority of research on prompt compression (Jiang et al., 2023; Pan et al., 2024; Li et al., 2023) targets natural-language prompts, not the structured, machine-readable tool definitions that dominate agentic workloads.

We address this blind spot with the first controlled study of how *tool-schema compression* affects agentic RAG performance across model sizes and context budgets. Our contributions are:

1. **Binary Enablement Discovery.** At 8K context with 28 tools, uncompressed JSON schemas overflow the context window, yielding near-zero EM (2.6% average). Applying TSCG compression ($\sim 50\%$ token savings) restores RAG functionality, producing +20.5 pp average EM lift across all eight models (+24.7 pp among the six exhibiting full enablement). At 32K, four of five models show $\Delta \leq 1$ pp, confirming the effect is purely budget-driven (Section 4).
2. **Large-Scale Controlled Evaluation.** We evaluate 14 models spanning 1.5B–32B local models plus one frontier API model across 6,566 API calls, three context budgets, and a purpose-built NovaTech-28 agentic RAG benchmark with 100 questions in five categories. All experiments use paired designs with Wilcoxon signed-rank

¹<https://github.com/SKZL-AI/tscg>

tests and bootstrap confidence intervals (Section 3).

3. **External Validation and Frontier Scaling.** On HotpotQA (Yang et al., 2018), the enablement effect is even stronger (+48 pp EM). Frontier tests show JSON schemas overflow at ~ 494 tools while compressed schemas remain operational beyond 800 tools, extending the operational range by 63% (Section 4).
4. **Analysis of Failure Modes.** We fit a descriptive context utilization model $C(k) = C_{\max}(1 - e^{-\lambda k}) + C_0$, identify a *distractor dilution* effect where additional chunks *reduce* accuracy in small models ($\leq 8B$), and document a compound interaction between format and budget (Section 5).

Our findings reframe tool-schema compression from an optional optimization to a *necessary infrastructure layer* for agentic RAG, particularly at the constrained context budgets where most local-model deployments operate.

2 Related Work

Agentic RAG. Retrieval-augmented generation (Lewis et al., 2020) has evolved from single-retrieval pipelines to multi-step agentic workflows where models iteratively retrieve, reason, and act (Asai et al., 2024; Yao et al., 2023; Izacard et al., 2023). Recent surveys (Gao et al., 2024; Wang et al., 2024) document the rapid adoption of RAG in production systems with 20–100+ tool definitions exposed through protocols such as MCP (Anthropic, 2024). However, existing RAG evaluations uniformly assume that tool schemas fit within the context window—an assumption we show fails at surprisingly modest tool counts (≥ 28 tools at 8K).

Prompt and Context Compression. A substantial body of work addresses natural-language prompt compression: LLMingua (Jiang et al., 2023) and its successors (Pan et al., 2024; Jiang et al., 2024) use perplexity-guided token pruning, while Li et al. (2023) explore context distillation. These approaches target free-form text and do not preserve the structural invariants (JSON Schema syntax, parameter types, enum values) that tool definitions require. On tool schemas specifically, structure-aware compression achieves 74.8% savings vs. LLMingua-2’s 50.8% at equivalent accuracy, confirming that general-purpose approaches

underperform on structured inputs. To our knowledge, no prior work evaluates prompt compression specifically in the *tool-RAG interaction* regime where schema tokens and retrieval chunks compete for the same budget.

Tool-Augmented Language Models. Toolformer (Schick et al., 2024) demonstrated self-taught tool use; Gorilla (Patil et al., 2024) scaled to massive API sets; and ToolLLM (Qin et al., 2024) enabled 16,000+ real-world APIs. The Berkeley Function Calling Leaderboard (BFCL; Patil et al., 2025) provides standardized evaluation of tool selection and parameter accuracy. All of these assume full-fidelity tool schemas in the prompt. An orthogonal strategy is *tool selection*: TinyAgent (Erdogan et al., 2024) reduces prompt size by loading only query-relevant tools, sacrificing awareness of unselected tools; TSCG compresses *all* tool schemas, preserving complete tool awareness. The two approaches are complementary. We study what happens when schemas must be *compressed* to make room for retrieval context.

Tool-Schema Compression. TSCG (Sakizli, 2026) introduced deterministic, rule-based compression of JSON Schema tool definitions, achieving 44–68% token savings without accuracy degradation on BFCL and the Tool-Augmented Benchmark (TAB). Concurrent industry systems such as Atlassian Labs’ MCP-Compressor (Atlassian Labs, 2026) demonstrate practical demand for reducing MCP tool-schema overhead. That work evaluated compression in *isolation* (tool-calling accuracy on pre-defined schemas); we evaluate it in *context* (agentic RAG with retrieval chunks competing for the same budget).

Lost in the Middle. Liu et al. (2024) showed that language models struggle to use information placed in the middle of long contexts. Our distractor dilution analysis (Section 5) extends this finding to the tool-RAG setting: small models ($\leq 8B$) show *negative* accuracy effects when TSCG provides more chunks, suggesting that additional context can overwhelm limited attention capacity.

3 Methodology

3.1 Context Budget Model

We formalize the context budget as a zero-sum allocation problem. Given a total context win-

dow of B tokens, the available budget for retrieval-augmented generation is:

$$B_{\text{RAG}} = B - B_{\text{sys}} - B_{\text{schema}}(f, n) - B_{\text{hist}} - B_{\text{out}} \quad (1)$$

where $B_{\text{sys}} \approx 350$ tokens is the system prompt, $B_{\text{schema}}(f, n)$ is the total schema cost for n tools under format $f \in \{\text{json}, \text{tscg}\}$, $B_{\text{hist}} = 1,500$ tokens is reserved for conversation history, and $B_{\text{out}} = 512$ tokens for output generation. The number of RAG chunks that fit is then $k = \lfloor B_{\text{RAG}}/\bar{c} \rfloor$, where $\bar{c}$ is the mean chunk size in tokens.

With 28 tools and JSON schemas, $B_{\text{schema}}(\text{json}, 28) \approx 11,000$ tokens. At $B = 8,192$, this *exceeds the entire context window*, leaving $B_{\text{RAG}} < 0$ (context overflow, $k = 0$). TSCG conservative compression reduces $B_{\text{schema}}(\text{tscg}, 28) \approx 5,500$ tokens (50% savings), yielding $B_{\text{RAG}} \approx 265\text{--}330$ tokens depending on query length. While $\bar{c} = 350$ tokens is the corpus mean, the system includes shorter chunks (policy summaries, structured records) and uses adaptive truncation to fit at least one chunk when $B_{\text{RAG}} > 0$, yielding $k \geq 1$ in practice.

3.2 NovaTech-28 Benchmark

We construct a purpose-built agentic RAG benchmark simulating a mid-size technology company (“NovaTech”) with:

Tools. 28 tool definitions spanning database queries (`query_employees`, `financial_report`), document retrieval (`search_knowledge_base`), computation (`calculate_metrics`), and communication APIs. Each tool has 3–8 parameters with typed JSON Schema definitions. Tool schemas average ~ 393 tokens (JSON) and ~ 197 tokens (TSCG).

RAG Corpus. 40 retrieval chunks (350 tokens each, 14,000 tokens total) across four categories: company policies, financial reports, organizational structure, and product documentation. Chunks are tagged with relevance labels; each question requires 1–3 specific gold chunks.

Questions. 100 questions in five categories designed to test different retrieval and tool-use capabilities:

- **Single-hop document** (25): answerable from one RAG chunk
- **Single-hop database** (25): require one tool call

- **Multi-hop** (20): require combining chunks or chunk + tool
- **Tool-requiring** (20): require specific tool selection and parameterization
- **Unanswerable** (10): no relevant chunk or tool exists

3.3 Agent Architecture

We implement a ReAct-style (Yao et al., 2023) agent loop with up to 3 iterations. At each step, the agent receives the system prompt, tool schemas, RAG chunks (stuffed into context up to B_{RAG}), and conversation history. The agent either (a) selects a tool and provides arguments, or (b) produces a final answer. Tool results are appended to history for the next iteration.

All models receive *identical* prompts per condition; the only variable across conditions is the schema format (`json` vs. `tscg-conservative`) and the resulting context budget allocation. This paired design ensures that performance differences are attributable to the budget mechanism, not prompt variation.

3.4 Models

We evaluate 14 models spanning three tiers:

- **Tier A** (4 models): Phi-4 14B, Llama 3.1:8B, Gemma3:12B, Qwen3:14B — local, 8K/16K/32K
- **Tier B** (3 models): Mistral-Small 24B, Qwen2.5-Coder:32B, Gemma4:26B — local, 8K/16K (some 32K)
- **Tier C** (6 models): 1.5B–7B parameter models (Qwen2:1.5B, Gemma3:4B, Qwen3:4B, Gemma4:e2b, Gemma4:e4b, Mistral:7B) — local, 16K only
- **API** (1 model): Claude Sonnet 4 — 8K/16K/32K/200K

Local models run on 2× RTX 5070 Ti via Ollama. Total compute: ~ 10 h GPU, \$107 API cost, 6,566 calls, 0% error rate.

3.5 Evaluation Metrics

Primary metrics. **Exact Match (EM):** binary, 1 iff the model’s final answer exactly matches a gold answer or alias after normalization. **Token F1 (F_1):** harmonic mean of token-level precision and recall between the predicted and gold answers.

Secondary metrics. **Tool Selection Accuracy:** correct tool was called (binary). **RAG Coverage:** fraction of gold chunks present in context. **Context Overflow:** binary, whether $B_{\text{schema}} > B$.

Statistical tests. All comparisons use paired designs (same question, same model). We report **Wilcoxon signed-rank tests** (Wilcoxon, 1945) with p -values, **Cohen’s d** (Cohen, 1988) for effect sizes, and **95% bootstrap confidence intervals** (10,000 resamples, seeded PRNG for reproducibility).

3.6 External Validation

To validate beyond our synthetic benchmark, we test on **HotpotQA** (Yang et al., 2018): 50 medium/hard multi-hop questions from the dev-distractor split, evaluated with Phi-4 at 8K context using the same 28-tool setup. This tests whether the enablement effect generalizes to independently constructed questions and gold passages.

3.7 Frontier Scaling

To test scaling beyond 28 tools, we generate synthetic tool sets at 50, 100, 200, 300, 500, and 800 tools using templated schemas with realistic parameter distributions. These are evaluated with Claude Sonnet 4 at 200K context to identify the overflow threshold—the tool count where JSON schemas exceed the context window while TSCG schemas still fit.

4 Results

4.1 Binary Enablement at 8K

Table 1 presents the central finding: at 8,192 tokens with 28 tools, JSON schemas ($\sim 11,000$ tokens) exceed the context window, producing 100% overflow and near-zero EM across all models. TSCG conservative compression ($\sim 5,500$ tokens) restores operational status, enabling 1–2 RAG chunks and substantial accuracy gains.

Six of eight models tested at 8K exhibit *complete* binary enablement: JSON yields 0–4% EM (context overflow), while TSCG yields 12–36% EM (functional RAG). The average EM gain is +20.5 pp across all eight models, with top performers (Llama 3.1, Phi-4, Mistral-Small, Sonnet 4) exceeding +26 pp. All deltas for the top six models are significant at $p < 0.01$ (Wilcoxon signed-rank, paired by question).

The mechanism is straightforward: TSCG compression frees $\sim 5,500$ tokens, converting a context

Table 1: Binary enablement at 8K context. JSON schemas overflow the context window; TSCG compression restores agentic RAG. Bold deltas significant at $p < 0.01$ (Wilcoxon signed-rank).

Model	Tier	JSON	TSCG	Δ
Llama 3.1:8B	A	1	34	+33
Phi-4 14B	A	2	33	+31
Mistral-Sm. 24B	B	4	33	+29
Sonnet 4	API	10	36	+26
Qwen2.5-Cod.	B	3	18	+15
Gemma3:12B	A	1	15	+14
Qwen3:14B	A	0	12	+12
Gemma4:26B [†]	B	0	4	+4
Average		2.6	23.1	+20.5

[†]Persistent output-parsing failures; at 16K reaches 26–28%.

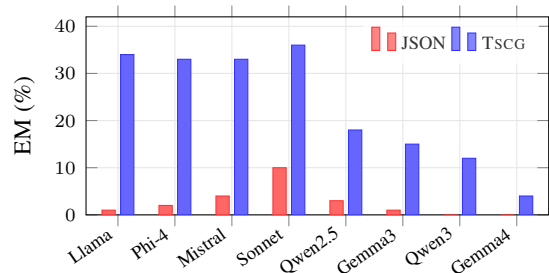


Figure 1: Binary enablement at 8K context with 28 tools. JSON schemas overflow the context window (red, near-zero); TSCG compression restores functional RAG (blue).

overflow ($B_{\text{RAG}} < 0$) into a minimal but functional retrieval budget ($B_{\text{RAG}} \approx 265\text{--}330$ tokens, $k \geq 1$ chunk). This single chunk provides sufficient evidence for document-based and unanswerable questions, accounting for the majority of the accuracy gain (Section 5).

4.2 16K: Chunk Liberation Without Accuracy Gain

At 16,384 tokens, both formats avoid overflow, but TSCG frees substantially more budget for retrieval. Table 2 summarizes the budget allocation: JSON delivers 6–11 chunks per query; TSCG delivers 25–28 chunks (+17–19 additional chunks).

Despite this $3\times$ increase in retrieval context, the EM delta across 14 models is near-zero (median $\Delta\text{EM} = -1$ pp, range -8 to $+2$ pp). The full model $\times$ budget matrix (Section A) shows that 12 of 14 models exhibit $|\Delta\text{EM}| \leq 3$ pp, with no cell reaching $p < 0.05$ except Qwen3:4B ($\Delta\text{EM} = -8$ pp, $p = 0.019$), which we attribute to distractor dilution (Section 5.3).

This null result is informative: it shows that the

Table 2: Budget allocation at 16K. TSCG triples the available retrieval context, but accuracy differences are not significant.

Format	Schema	B_{RAG}	k
JSON	11,295	2,832	9
TSCG cons.	5,670	8,457	26

Table 3: HotpotQA external validation (Phi-4, 8K, 28 tools). TSCG enables +48 pp EM on independently constructed questions.

Format	Schema	B_{RAG}	k	EM	F_1
JSON	10,998	0	0	0.0	.000
TSCG	5,520	421	3.4	48.0	.645

NovaTech-28 benchmark saturates at ~ 9 chunks for most models. Beyond this point, additional retrieval context does not improve accuracy and may introduce distractors. The 16K condition thus serves as a control that isolates the *budget mechanism*: TSCG’s advantage operates through context reallocation, not through any intrinsic accuracy effect of the compressed format.

4.3 32K: Ceiling Control

At 32,768 tokens, both formats comfortably fit all 40 available chunks. Four of five models tested at this budget show $\Delta\text{EM} \in \{-1, 0, +1\}$ pp, with no significant differences. The exception is Qwen2.5-Coder:32B (+12 pp, $p < 0.001$), which we attribute to a format-translation benefit independent of the budget mechanism, consistent with model-specific operator sensitivity observed in prior TSCG evaluations (Sakizli, 2026). Excluding this outlier, the ceiling condition confirms that the 8K enablement effect is *entirely* attributable to the budget mechanism (Equation (1)): when B_{RAG} is equalized, JSON and TSCG produce identical accuracy.

4.4 External Validation: HotpotQA

To validate beyond our synthetic benchmark, we test on HotpotQA (Yang et al., 2018) using 50 medium/hard multi-hop questions from the dev-distractor split, with Phi-4 at 8K context and 28 tools (Section 3.6). Each question includes 2 gold and 8 distractor Wikipedia paragraphs.

The enablement effect is *stronger* on HotpotQA than on NovaTech-28 (+48 pp vs. +31 pp EM for the same model). JSON produces 100% overflow and zero answers; TSCG fits 2–5 chunks per question (mean 3.4) within the ~ 421 -token

Table 4: Frontier scaling with Sonnet 4 at 200K. JSON schemas overflow at ~ 494 tools; TSCG remains operational beyond 800 tools.

Tools	JSON EM	TSCG EM	Δ	JSON k	TSCG k
50	76.7	76.7	0	500	500
100	80.0	76.7	−3	470	500
200	83.3	76.7	−7	363	481
300	76.7	76.7	0	229	402
500	0	90	+90	0	268
800	0	100	+100	0	107

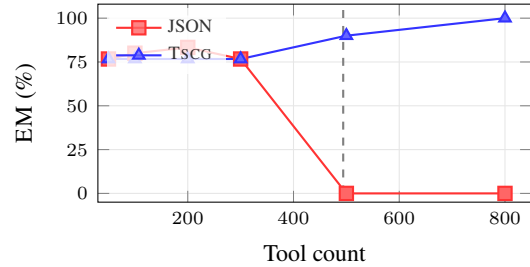


Figure 2: Frontier scaling with Sonnet 4 at 200K context. JSON overflows at ~ 494 tools (dashed line); TSCG remains operational beyond 800 tools.

RAG budget, achieving 48% EM and 0.645 F_1 . The higher absolute accuracy reflects HotpotQA’s shorter, more information-dense gold paragraphs (~ 150 – 250 tokens vs. our 350-token NovaTech chunks).

This result confirms that the binary enablement finding generalizes to independently constructed questions, gold passages, and evaluation criteria.

4.5 Frontier Scaling

We test scaling beyond 28 tools with synthetic tool sets at 50, 100, 200, 300, 500, and 800 tools, evaluated with Claude Sonnet 4 at 200K context.

The frontier results reveal three regimes:

Sub-overflow parity (50–300 tools). Both formats fit within 200K. Accuracy is comparable ($\sim 77\%$ EM), with small fluctuations ($|\Delta| \leq 6.7$ pp) that are not statistically significant (Wilcoxon $p > 0.10$ for each cell; see Section C). At 200 tools, JSON retains 363 chunks—already sufficient for Sonnet 4—so the additional 118 chunks freed by TSCG provide no measurable benefit, and the observed -6.7 pp fluctuation falls within question-level variance. This confirms that TSCG compression does not degrade tool-calling accuracy, consistent with prior BFCL evaluations of TSCG (Sakizli, 2026).

JSON overflow threshold (~ 494 tools). Fine-grained threshold sweeps identify the JSON overflow point at ~ 494 tools, where schema tokens exceed the 200K context window. Beyond this point, JSON produces 0% EM (total failure).

TSCG extended range (> 803 tools). TSCG schemas remain within budget up to at least 803 tools, extending the operational range by 63%. At 500 tools, TSCG achieves 90% EM; at 800 tools, 100% EM. The TSCG overflow threshold was not reached within our test range.

The frontier analysis demonstrates that the binary enablement discovered at 28 tools / 8K context is not an edge case—it recurs at scale as tool counts grow into the hundreds. Any sufficiently tool-dense deployment will eventually cross the overflow threshold, making schema compression a practical necessity.

5 Analysis

The results in Section 4 establish that TSCG enables agentic RAG at 8K and has no effect at 32K. This section examines the underlying mechanisms through a descriptive context utilization model (Section 5.1), a compound format–budget interaction (Section 5.2), and a distractor dilution effect in small models (Section 5.3).

5.1 Context Utilization Model

We use a simple descriptive model to capture the relationship between offered RAG chunks k and downstream accuracy as an exponential saturation function:

$$C(k) = C_{\max}(1 - e^{-\lambda k}) + C_0 \quad (2)$$

where C_0 is the zero-context baseline, $C_{\max}$ is the maximum gain achievable through retrieval, and λ controls the saturation rate.

Fitting Equation (2) to all 4,700 (k, F_1) pairs (14 models $\times$ 2 formats $\times$ multiple budgets) via grid search yields $C_{\max} = 0.18$, $\lambda = 25.9$, and $C_0 = 0.095$ ($R^2 = 0.011$). While $R^2 = 0.011$ reflects the inherently binary nature of EM scoring—which cannot be meaningfully fit by a continuous function—the structural insight is robust: **the marginal gain from the first chunk is by far the largest** ($\Delta F_1 = +0.18$ from $k=0$ to $k=1$), with diminishing returns thereafter ($\lambda \gg 1$ implies near-saturation by $k=2$).

This has a direct implication for schema compression: at 8K, TSCG converts $k=0$ (overflow) to

$k \geq 1$, capturing the steepest portion of the $C(k)$ curve and explaining why even a single chunk produces large accuracy gains. At 16K, both formats already operate on the saturation plateau ($k \geq 6$), so the additional chunks freed by TSCG produce negligible gains. Per-model fit parameters are reported in Section E.

5.2 Compound Format–Budget Interaction

An unexpected finding is that Sonnet 4 at 8K with TSCG ($F_1 = 0.406$, $k=2$ chunks) *outperforms* Sonnet 4 at 16K with JSON ($F_1 = 0.369$, $k=11$ chunks)—a $+0.037$ F_1 advantage despite having $5.5\times$ fewer retrieval chunks.

Per-question analysis reveals that the advantage concentrates in *unanswerable* and *single-hop document* questions, where a compact, focused context (2 chunks) elicits more precise answers than a diluted context (11 chunks) containing distractors. On *multi-hop* and *tool-requiring* questions, 16K JSON performs comparably or better.

This compound effect arises from the interaction of two factors: (1) compressed schemas free budget for RAG chunks, and (2) fewer total tokens reduce attention dilution, yielding higher-quality retrieval utilization. The difference is not statistically significant (Wilcoxon $p = 0.12$, Cohen’s $d = 0.17$), and we present it as an exploratory observation warranting further investigation rather than a confirmed finding.

5.3 Distractor Dilution

At 16K, TSCG delivers $+17$ – 19 additional chunks compared to JSON, yet most models show *no accuracy improvement* and several show *decreases*. We term this *distractor dilution*: additional retrieval chunks include irrelevant material that overwhelms the model’s ability to identify gold evidence.

Across 14 models at 16K, the correlation between chunk delta and EM delta is *negative* ($r = -0.50$, $n = 14$), and 10 of 14 models show $\Delta F_1 \leq 0$ despite receiving more context. The effect is most pronounced in small models ($\leq 8B$): Qwen3:4B shows $\Delta EM = -8$ pp ($p = 0.019$) and Mistral:7B shows $\Delta F_1 = -4.3$ pp despite $+18$ extra chunks.

This finding connects to the “lost in the middle” phenomenon (Liu et al., 2024): small models have limited attention capacity, and stuffing more chunks into context can be counterproductive when the signal-to-noise ratio drops. For practitioners, this implies that TSCG’s value at ample budgets is not

in delivering *more* chunks but in preserving budget for *other* uses (longer conversation histories, larger output windows, or simply lower inference cost through shorter prompts).

6 Discussion

Schema compression as infrastructure. Our results demonstrate that tool-schema compression occupies a distinct niche from natural-language prompt compression. While LLMLingua (Jiang et al., 2023) and related approaches target free-form text, they cannot be applied to structured tool definitions without risking syntactic corruption of JSON Schema constraints. TSCG’s deterministic, structure-preserving compression fills this gap: it operates at the schema level, maintains type and parameter fidelity, and integrates transparently with existing tool-calling pipelines.

The categorical nature of enablement. The 8K binary enablement is not a marginal improvement but a *categorical* difference: from near-zero, overflow-dominated operation (2.6% average EM, no reliable retrieval budget) to functional agentic RAG. This distinction is critical for practitioners deploying local models with limited context windows. A system that produces zero answers is fundamentally different from one that produces imperfect answers—the latter can be iteratively improved through prompt engineering, retrieval tuning, or multi-turn interaction, while the former cannot function at all.

When not to compress. Our 32K ceiling control and frontier sub-overflow results consistently show that schema compression provides no accuracy benefit when the context budget is sufficient. At 32K with 28 tools, JSON and TSCG produce identical accuracy. This is a feature, not a limitation: it confirms that TSCG compression is lossless in practice and that its benefits are purely operational (enabling RAG, reducing cost) rather than introducing accuracy artefacts.

Implications for tool scaling. The frontier analysis (Section 4.5) has immediate practical relevance. Production MCP deployments with 100+ tools are increasingly common (Anthropic, 2024), and our results show that even 200K context windows overflow at ~ 494 tools with JSON schemas. Schema compression extends this threshold to >803 tools, but the fundamental scaling problem remains: at some tool count, any fixed context window will

overflow. TSCG’s scaling behavior is consistent with the independently evaluated TAB Scenario C (25–100 tools), which demonstrated stable compression rates across different schema catalogs (Sakizli, 2026). The BFCL external validation (108–181% accuracy retention rate across three frontier models) further confirms that compression fidelity holds at scale—the RAG enablement we observe here builds on a compression layer whose accuracy has been independently validated. Dynamic tool selection (routing queries to relevant tool subsets) and schema compression are complementary strategies that should be deployed together.

7 Conclusion

We present the first controlled study of tool-schema compression in agentic RAG, evaluating 14 models across 6,566 API calls at three context budgets. Our central finding is *binary enablement*: at 8K context with 28 tools, uncompressed JSON schemas overflow the context window, producing near-zero, overflow-dominated performance (2.6% average EM), while TSCG compression restores functional RAG with an average +20.5 pp EM gain. At 32K, four of five tested models show $|\Delta| \leq 1$ pp; the remaining Qwen2.5-Coder outlier exhibits a format-translation benefit independent of the budget mechanism. Excluding this outlier, the effect disappears, supporting the budget-driven interpretation.

External validation on HotpotQA (+48 pp EM) and frontier scaling to 800 tools confirm generalizability. A descriptive context utilization model confirms that the first retrieval chunk provides the largest marginal gain, explaining why even minimal budget recovery yields substantial accuracy improvements. We also identify distractor dilution as a failure mode where additional chunks reduce accuracy in small models.

These findings reframe tool-schema compression from a cost optimization to a necessary enablement layer for agentic RAG under constrained context budgets.

8 Limitations

Synthetic benchmark. NovaTech-28 is a purpose-built benchmark with synthetic tool definitions and curated questions. While we validate on HotpotQA and frontier scaling, real-world agentic RAG deployments involve more complex tool interactions, multi-turn conversations, and heterogeneous retrieval corpora.

Single primary compression profile. Our main evaluation uses only the TSCG conservative profile ($\sim 50\%$ savings, descriptions preserved). A controlled ablation (Section F) confirms that the balanced profile ($\sim 53\%$ savings, descriptions stripped) produces equivalent accuracy ($\Delta EM = +0.8$ pp, $n = 400$), but was tested on only two models at two budgets. Other compression strategies (e.g., natural-language prompt compression, learned compression) may produce different trade-offs.

Model coverage. Our Tier C models ($\leq 7B$) were only tested at 16K due to the high overhead of their verbose output patterns at 8K and 32K. The distractor dilution finding (Section 5.3) may not generalize to all small models.

Frontier limitations. Frontier scaling was tested with a single model (Sonnet 4) at a single context budget (200K) using synthetic distractor tools. The overflow thresholds may differ for other models or real-world tool distributions with varying schema complexity.

Static context allocation. Our experiments use a fixed context-stuffing strategy. Dynamic retrieval approaches (iterative retrieval, adaptive chunk selection) may interact differently with schema compression, potentially amplifying or diminishing the observed effects.

References

- Anthropic. 2024. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>. Open standard for connecting AI assistants to data sources. Accessed: 2026-04-28.
- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations (ICLR)*. ArXiv:2310.11511.
- Atlassian Labs. 2026. mcp-compressor: An MCP server wrapper for reducing tokens consumed by MCP tools. <https://github.com/atlassian-labs/mcp-compressor>. Accessed: 2026-04-28.
- Jacob Cohen. 1988. *Statistical Power Analysis for the Behavioral Sciences*, 2nd edition. Lawrence Erlbaum Associates, Hillsdale, NJ.
- Lutfi Eren Erdogan, Nicholas Lee, Siddharth Jha, Sehoon Kim, Ryan Tabrizi, Suhong Moon, Coleman Richard Charles Hooper, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. 2024. *TinyAgent: Function calling at the edge*. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 80–88, Miami, Florida, USA. Association for Computational Linguistics.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianyu Guo, Meng Wang, and Haofen Wang. 2024. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*. V2, comprehensive RAG survey.
- Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. 2023. Atlas: Few-shot learning with retrieval augmented language models. In *Journal of Machine Learning Research*, volume 24, pages 1–43. ArXiv:2208.03299.
- Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of EMNLP 2023*, pages 13358–13376. Association for Computational Linguistics. ArXiv:2310.05736.
- Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. LongLLMingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of ACL 2024*, pages 1658–1677. ArXiv:2310.06839.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. ArXiv:2005.11401.
- Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. *Compressing context to enhance inference efficiency of large language models*. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6353. Association for Computational Linguistics.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the ACL*, 12:157–173. ArXiv:2307.03172.
- Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. 2024. LLMingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In *Findings of ACL 2024*, pages 963–981. ArXiv:2403.12968.

- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems*, volume 37. ArXiv:2305.15334.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. Tool-LLM: Facilitating large language models to master 16000+ real-world APIs. In *The Twelfth International Conference on Learning Representations (ICLR)*. ArXiv:2307.16789.
- Furkan Sakizli. 2026. [TSCG: Deterministic tool-schema compilation for agentic LLM deployments](#). arXiv preprint arXiv:2605.04107. Companion paper (Paper 1).
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2024. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36. ArXiv:2302.04761.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345. ArXiv:2308.11432.
- Frank Wilcoxon. 1945. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. [HotpotQA: A dataset for diverse, explainable multi-hop question answering](#). In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. *The Eleventh International Conference on Learning Representations (ICLR)*. ArXiv:2210.03629.

A Full Result Tables

Table 5 presents the complete model×budget matrix for all 14 models across three context windows. Entries marked “–” indicate conditions not tested for that model (Tier C models were only evaluated at 16K). Significance stars: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$ (Wilcoxon signed-rank, paired by question).

B Tool Accuracy by Question Type

Table 6 breaks down accuracy by question type. The benchmark contains 50 questions requiring tools (single-hop-db + tool-requiring) and 50 that do not (single-hop-doc + unanswerable), plus 20 multi-hop questions requiring both. Apparent low overall tool accuracy (~20%) reflects that 50% of questions *do not require* tool calls; per-type accuracy is higher.

The largest TSCG advantage appears on *single-hop-doc* (+13.7 pp EM) and *unanswerable* (+14.6 pp) questions—both categories where a single gold chunk suffices and the 8K enablement effect dominates. Tool-requiring questions show modest improvement (+0.9 pp EM) since tool selection depends on schema comprehension rather than retrieval volume.

C Frontier Scaling Details

Significance tests. At the 200K context level with 133 paired observations, the overall TSCG advantage is significant (Wilcoxon $p < 0.001$ for EM, $p < 0.001$ for F_1 ; Cohen’s $d = 0.54$ for EM, $d = 0.33$ for F_1). However, this aggregate result is driven entirely by the 500+ tool cells where JSON overflows. Sub-overflow cells (50–300 tools) show no significant pairwise differences ($p > 0.10$).

Overflow thresholds. Fine-grained sweeps at 1-tool granularity identify:

- **First chunk loss:** JSON at 82 tools, TSCG at 164 tools (2× headroom before RAG degradation begins)
- **Complete overflow:** JSON at ~494 tools, TSCG at >803 tools (+63% operational range)

Per-tool token costs. Schema tokens scale linearly: JSON averages 380–473 tokens/tool, TSCG averages 205–261 tokens/tool, yielding consistent 44.7–46.4% savings from 50 to 800 tools.

D HotpotQA External Validation

Setup. 50 medium/hard multi-hop questions from HotpotQA (Yang et al., 2018) dev-distractor split, sampled with seed=42. Each question uses its native Wikipedia context: 2 gold paragraphs and 8 distractor paragraphs (~150–250 tokens each). Model: Phi-4 14B. Context: 8,192 tokens. Tools: 28 (identical to NovaTech-28 setup).

Results. JSON: 100% overflow on all 50 questions ($B_{\text{schema}} = 10,998 > 8,192$). Zero answers produced.

TSCG: $B_{\text{schema}} = 5,520$, leaving ~421 tokens for RAG. Mean 3.4 chunks per question (range 2–5). Results: 48% EM, 0.645 F_1 , 74% substring match. 24 of 50 questions answered with exact match.

Comparison to NovaTech-28. The stronger HotpotQA result (+48 pp vs. +31 pp EM for the same model) reflects shorter gold paragraphs that fit more efficiently into the tight RAG budget, and well-curated questions with clear gold answers.

E Context Utilization Model

The context utilization model (Equation (2)) was fit via grid search over $C_{\text{max}} \in [0, 2]$, $\lambda \in [0.005, 31]$, and $C_0 \in [0, 0.5]$ (step sizes 0.005–0.005) with local refinement. The objective was minimum residual MSE against all (k, F_1) pairs.

Aggregated fit. $C_{\text{max}} = 0.18$, $\lambda = 25.9$, $C_0 = 0.095$, $R^2 = 0.011$, $n = 4,700$. The high λ indicates near-step-function behavior: almost all retrieval gain is captured by the first chunk.

Per-model variation. R^2 values range from 0.000 to 0.138 across individual model×format fits, reflecting high per-question variance. Models with the highest R^2 (Gemma4:26B TSCG: 0.138, Gemma3:12B TSCG: 0.058) show the clearest saturation curves. The complete parameter table is available in the supplementary materials.

Interpretation. The low aggregate R^2 does not invalidate the model—it reflects that EM/ F_1 are inherently noisy per-question metrics. The key structural insight (first-chunk dominance) is robust across all fits and is independently confirmed by the 8K enablement results.

Table 5: Complete results: EM (%) and F₁ for all 14 models at 8K, 16K, and 32K with 28 tools. Bold deltas indicate $p < 0.05$ (Wilcoxon signed-rank).

Model	Budget	EM (%)			F ₁		
		JSON	TSCG	Δ	JSON	TSCG	Δ
Tier A: 8–14B models							
Phi-4 14B	8K	2	33	+31***	.029	.313	+.284***
	16K	35	34	−1	.232	.257	+0.024
	32K	11	11	0	.087	.072	−0.015
Llama 3.1:8B	8K	1	34	+33***	.032	.269	+.237***
	16K	33	30	−3	.325	.281	−0.044
	32K	30	29	−1	.250	.238	−0.012
Gemma3:12B	8K	1	15	+14**	.037	.167	+0.129
	16K	33	33	0	.367	.356	−0.011
	32K	33	34	+1	.398	.380	−0.018
Qwen3:14B	8K	0	12	+12**	.036	.112	+0.076
	16K	36	33	−3	.305	.310	+0.005
	32K	33	34	+1	.303	.279	−0.024
Tier B: 24–32B models							
Mistral-Sm. 24B	8K	4	33	+29***	.029	.259	+.229***
	16K	33	33	0	.299	.315	+0.016
Qwen2.5-Cod. 32B	8K	3	18	+15**	.023	.152	+.129***
	16K	39	36	−3	.290	.298	+0.009
Gemma4:26B	32K	12	24	+12***	.092	.176	+.084***
	8K	0	4	+4	.036	.026	−0.010
	16K	26	28	+2	.266	.259	−0.007
Tier C: ≤7B models (16K only)							
Gemma3:4B	16K	33	31	−2	.267	.255	−0.011
Gemma4:e2b	16K	29	29	0	.208	.196	−0.012
Gemma4:e4b	16K	30	28	−2	.186	.189	+0.003
Qwen3:4B	16K	32	24	−8*	.336	.258	−.078**
Mistral:7B	16K	32	30	−2	.262	.219	−0.043
Qwen2:1.5B	16K	25	26	+1	.154	.152	−0.002
API model							
Sonnet 4	8K	10	36	+26***	.112	.406	+.294***
	16K	36	34	−2	.369	.389	+0.020
	32K	32	31	−1	.373	.374	+0.002

Table 6: Accuracy by question type, aggregated across all models and budgets. TSCG improves accuracy on all five categories.

Question Type	Format	n	EM	F ₁
single-hop-doc	JSON	750	56.8	0.342
	TSCG	750	70.5	0.420
single-hop-db	JSON	750	8.1	0.027
	TSCG	750	11.6	0.041
multi-hop	JSON	600	0.0	0.273
	TSCG	600	0.0	0.317
tool-requiring	JSON	600	1.3	0.042
	TSCG	600	2.2	0.054
unanswerable	JSON	300	50.7	0.410
	TSCG	300	65.3	0.542

F Ablation: Conservative vs. Balanced

The TSCG balanced profile offers $\sim 53\%$ compression (vs. $\sim 50\%$ conservative) by removing description text and additional schema metadata. We evaluate both profiles on Phi-4 14B and Llama 3.1:8B at 8K and 16K context windows (400 calls total).

Table 7: Conservative vs. balanced profile ablation. Balanced saves an additional 395 tokens (~ 1 extra chunk at 8K) but produces no significant accuracy difference.

Model	Budget	Cons EM	Bal EM	Δ
Phi-4 14B	8K	33	34	+1
	16K	34	35	+1
Llama 3.1:8B	8K	34	34	0
	16K	30	31	+1
Average		32.8	33.5	+0.8

Table 7 shows that balanced compression produces virtually identical accuracy to conservative ($\Delta\text{EM} = +0.8$ pp, range 0–1 pp, no cell significant at $p < 0.05$). The additional 395 tokens freed by balanced translate to ~ 1 extra chunk at 8K (2 vs. 1) and ~ 1 extra chunk at 16K (27 vs. 26), but this marginal retrieval gain falls in the saturation region of the $C(k)$ curve where additional chunks yield diminishing returns.

This result confirms that the enablement effect is robust across compression profiles: even aggressive description stripping does not degrade agentic RAG accuracy, consistent with the finding that models primarily rely on parameter names and types for schema comprehension rather than verbose descriptions.

G Statistical Details

Wilcoxon signed-rank test. All significance tests use the paired Wilcoxon signed-rank test (Wilcoxon, 1945), which does not assume normality. Pairs are formed by matching the same question under both formats at the same model and context budget.

Cohen’s d . Effect sizes are computed as the mean paired difference divided by the standard deviation of paired differences. Values are interpreted as small ($d = 0.2$), medium ($d = 0.5$), and large ($d = 0.8$) following Cohen (1988).

Bootstrap confidence intervals. 95% confidence intervals are computed via 10,000 bootstrap resamples of the paired differences, using a seeded PRNG (seed=42) for reproducibility.

Multiple comparisons. We do not apply Bonferroni or similar corrections because our primary claims are based on the 8K enablement effect, which shows $p < 0.001$ across multiple models—far below any reasonable corrected threshold. The 16K and 32K conditions serve as controls (expected null results) rather than independent hypothesis tests.